

Documentação de Arquitetura: Core API

Padronização de Camadas, Estrutura de Pastas e Clean Architecture

1. Visão Geral

Esta API adota princípios de **Clean Architecture** e **Domain-Driven Design (DDD)**. A estrutura foca no isolamento total da inteligência de negócio (Domínio) das tecnologias de transporte e infraestrutura.

2. Estrutura de Diretórios

```
core-api/  
├── src/  
│   ├── domain/           # Coração do Sistema (Regras de Negócio)  
│   │   ├── entities/     # Modelos Ricos com regras de estado (.entity.ts)  
│   │   ├── services/     # Lógica complexa multi-entidade (.service.ts)  
│   │   └── interfaces/   # Contratos e Interfaces de infra (.interface.ts)  
│   ├── v1/               # Camada de Aplicação (Versão 1 da API)  
│   │   ├── controllers/  # Gestão de entrada/saída HTTP (.controller.ts)  
│   │   ├── dtos/         # Validação e Contratos com Zod (.dto.ts)  
│   │   ├── middlewares/  # Filtros e Validadores (.middleware.ts)  
│   │   ├── routes/       # Definição de endpoints (.routes.ts)  
│   │   └── repositories/ # Implementações de Banco de Dados (.repository.ts)  
│   └── server.ts         # Ponto de entrada da aplicação  
└── package.json         # Dependências e Scripts
```

3. Responsabilidades das Camadas

3.1 Camada de Domínio (domain)

A camada mais interna e estável. Não deve conhecer detalhes de bibliotecas externas ou frameworks.

- **Entities:** Objetos que representam conceitos de negócio e garantem sua própria integridade.
- **Domain Services:** Realizam operações que envolvem mais de uma entidade ou dependem de abstrações externas.
- **Interfaces:** Contratos que definem o que o sistema precisa para funcionar (ex: Repositórios).

3.2 Camada de Aplicação (v1)

Responsável por orquestrar como o mundo exterior interage com o domínio.

- **DTOs:** Filtram dados de entrada usando **Zod**, garantindo "fail-fast".
- **Controllers:** Recebem requisições, chamam serviços e decidem a resposta HTTP.
- **Repositories:** Traduzem comandos de domínio para persistência física (SQL/ORM).

4. Padrões de Nomenclatura

Elemento	Padrão	Exemplo
Arquivos e Pastas	kebab-case (minúsculo)	<code>user-repository.ts</code>
Classes e Tipos	PascalCase	<code>class UserEntity</code>
Sufixos	Obrigatório por tipo	<code>.entity.ts</code> , <code>.controller.ts</code>

5. Fluxo de uma Requisição

Request HTTP → Rota → Middleware (Validação Zod) → Controller → Service (Aplicação) → Entity (Domínio) → Repository → Response JSON.

